

Numerical Computing on GPUs

Numerical computing on a GPU is not only about obtaining an answer; it is about obtaining an answer that is fast enough, accurate enough, reproducible enough, and representable in the chosen format. Optimized libraries such as cuBLAS, cuDNN, cuSOLVER, cuFFT, and framework kernels already encode years of work on tiling, memory access, instruction selection, and numerical behavior. Use them first for standard operations. Write a custom kernel only when the operation is unsupported, fusion removes meaningful memory traffic, or profiling proves that the library call is the bottleneck.

This guide develops the numerical ideas needed to choose library modes intelligently and explain the trade-offs in GPU-programming interviews.

Floating-Point Representation

1. Overview

Definition

Floating point represents a large range of real numbers using a sign, a significand (fraction), and a scale (exponent). Most CPUs and GPUs follow IEEE 754 semantics for common binary formats.

For a normal binary floating-point value:

$$\text{value} = (-1)^{\text{sign}} \times (1.\text{fraction}) \times 2^{(\text{exponent} - \text{bias})}$$

The bits are finite, so most real numbers—including 0.1—cannot be represented exactly.

Why it matters

- Rounding occurs after most arithmetic operations.
- Precision varies with magnitude; spacing is not uniform.
- Algebraically equivalent expressions can produce different machine results.
- Parallel GPU reductions may change the operation order and therefore the last bits.
- Format choice changes memory bandwidth, capacity, throughput, and accuracy.

Where it is used

Floating point is used in machine learning, simulations, graphics, signal processing, finance, linear algebra, image processing, and almost every GPU workload involving non-integer data.

Why interviewers ask

Interviewers want to know whether you can explain rounding, special values, comparisons, non-associativity, and why a mathematically correct parallel program may produce slightly different results from a CPU implementation.

2. Core Idea

Intuition and analogy

Scientific notation stores 6.02×10^{23} as a few significant digits plus an exponent. Binary floating point does the same in base 2. It behaves like a ruler whose tick spacing grows as you move away from zero: values near zero are close together, while large values are farther apart.

Small example

A toy format with three significant binary digits can represent:

```
1.002 = 1.0
1.012 = 1.25
1.102 = 1.5
1.112 = 1.75
```

The real value 1.3 lies between 1.25 and 1.5 , so it must be rounded. Once rounded, the missing information cannot be recovered by later operations.

Step-by-step: why $0.1 + 0.2 \neq 0.3$ can occur

1. 0.1 , 0.2 , and 0.3 have repeating binary expansions.
2. Each is rounded to the nearest representable value when stored.
3. Arithmetic uses those approximations, not the exact decimal values.
4. The rounded sum may differ by one or more units in the last place from the stored approximation of 0.3 .
5. Exact equality therefore tests bit-level equality, not mathematical closeness.

3. Important Subtopics

3.1 Sign, exponent, and fraction

- **Meaning:** The sign chooses positive or negative, the exponent controls scale, and the fraction controls precision.
- **Why it matters:** More exponent bits increase range; more fraction bits increase precision.
- **Example:** FP32 has 1 sign bit, 8 exponent bits, and 23 stored fraction bits. Normal values have 24 bits of effective significand precision because the leading 1 is implicit.
- **Interview angle:** Explain the range-versus-precision trade-off rather than only memorizing bit counts.

3.2 Normal and subnormal numbers

- **Meaning:** Normal values use an implicit leading 1 . Subnormals use an implicit leading 0 and fill the gap between the smallest normal value and zero.
- **Why it matters:** Subnormals provide **gradual underflow**, but some GPU modes flush them to zero for speed.
- **Example:** Repeatedly halving a small number passes through subnormal values before becoming zero when gradual underflow is preserved.

- **Interview angle:** Know that “smallest positive value” may mean smallest normal or smallest subnormal.

3.3 Zero, infinity, and NaN

- **Meaning:** IEEE 754 reserves exponent patterns for $+0$, -0 , $+\infty$, $-\infty$, and NaN.
- **Why it matters:** These values let exceptional arithmetic propagate without immediately trapping.
- **Example:** $1.0 / 0.0$ commonly produces infinity; $0.0 / 0.0$ produces NaN.
- **Interview angle:** NaN is unordered: comparisons such as $x == \text{NaN}$ are false; use an `isnan` test.

3.4 Rounding and unit in the last place

- **Meaning:** An exact result is mapped to a representable result. The common default is round-to-nearest, ties-to-even. An ULP measures spacing between adjacent values near a number.
- **Why it matters:** Error is naturally discussed in ULPs or relative error, not only decimal digits.
- **Example:** Adding a tiny value to a very large value may return the large value unchanged because the tiny increment is below half an ULP.
- **Interview angle:** Explain why machine epsilon is a relative spacing near 1 , not a universal absolute tolerance.

3.5 Non-associativity

- **Meaning:** Floating-point addition and multiplication are not generally associative.
- **Example:** In FP32, $(1e20 + -1e20) + 3$ is approximately 3 , while $1e20 + (-1e20 + 3)$ is approximately 0 because 3 is lost in the second grouping.
- **Why it matters:** GPU tree reductions use a different order from serial loops.
- **Interview angle:** Different final bits do not automatically indicate a race condition.

3.6 Fused multiply-add (FMA)

- **Meaning:** `fma(a, b, c)` computes $a \times b + c$ with one final rounding rather than rounding the product and then the sum.
- **Why it matters:** It is typically faster and more accurate, but can differ from unfused CPU reference results.
- **Example:** Dot products benefit because each multiply-add introduces only one rounding.
- **Interview angle:** Compiler contraction and hardware FMA can affect reproducibility.

4. Real-World Example

Consider a GPU dot product used inside a recommendation model:

```
score =  $\sum$  user_embedding[i]  $\times$  item_embedding[i]
```

An optimized BLAS library tiles the vectors, uses coalesced loads, performs FMA instructions, reduces partial sums in parallel, and chooses an accumulator type. A hand-written loop may use a different summation order or fail to exploit specialized matrix hardware. The library is normally the right choice; the developer still decides whether the accepted result tolerance and compute mode are appropriate.

5. Diagrams / Mental Models

FP32: | sign: 1 | exponent: 8 | fraction: 23 |
 direction scale precision

Exponent all 0s -> zero or subnormal

Exponent middle -> finite normal number

Exponent all 1s -> infinity or NaN

Representable values are unevenly spaced:

0 --|--|--|-- 1 ----|----|---- 2 -----|----- 4
 fine spacing wider spacing wider again

6. Common Interview Questions

#	Question and clear answer	Key points expected	Common mistake
1	Why cannot binary floating point represent 0.1 exactly? Its binary expansion repeats, but storage has finitely many fraction bits, so it is rounded.	Base-2 representation and finite precision.	Saying all decimals are inexact; powers of two and many fractions are exact.
2	What are the fields in IEEE floating point? Sign, biased exponent, and fraction/significand bits.	Range comes mainly from exponent; precision from significand.	Calling the stored fraction the entire mantissa without mentioning the implicit bit.
3	What is machine epsilon? Commonly, the gap from 1 to the next representable value in a format.	It describes relative precision near 1.	Using epsilon as one absolute comparison tolerance for every magnitude.
4	Why is addition non-associative? Each intermediate result is rounded, and regrouping changes which information is discarded.	Give a cancellation or large-plus-small example.	Claiming addition is random.
5	What is a NaN? A special value representing an undefined or invalid floating result; it propagates through much arithmetic.	isnan, unordered comparisons, payloads are optional detail.	Testing <code>x == NaN</code> .
6	What is a subnormal? A tiny value with no implicit leading 1, used for gradual underflow near zero.	Reduced precision, gap to zero, possible FTZ behavior.	Saying it has the same relative precision as a normal value.
7	What does FMA change? It evaluates multiply-plus-add with one rounding,	One rounding versus two.	Assuming the result must match separately

#	Question and clear answer	Key points expected	Common mistake
	normally improving accuracy and throughput.		rounded operations bit for bit.
8	Why can CPU and GPU answers differ? They may use different formats, FMA contraction, math approximations, and operation orders.	Difference should be bounded and tested with a suitable error metric.	Immediately blaming GPU hardware.
9	How should floating results be compared? Use a problem-appropriate combination of relative and absolute tolerance, with explicit handling for NaN and infinity.	Scale-aware comparison.	Exact equality for computed results or relative error alone near zero.
10	What does an ULP mean? It is the spacing of representable values at a particular magnitude; ULP error measures how many adjacent values separate results.	Spacing changes with exponent.	Treating one ULP as a fixed decimal quantity.

7. Deep-Dive Questions

- Why are there positive and negative zeros?** The sign bit remains meaningful when the magnitude is zero. They compare equal, but operations such as $1/+0$ and $1/-0$, `copysign`, and some branch-cut functions can distinguish them.
- Can $(a + b) + c$ be made reproducible across GPU runs?** Yes, but usually at a cost: fix the reduction tree, format, rounding behavior, compiler options, and library algorithm; avoid atomics whose arrival order varies. Cross-architecture bitwise reproducibility may need stronger constraints or exact/reproducible accumulation.
- What is catastrophic cancellation?** Subtracting nearly equal approximations removes leading significant bits, exposing earlier rounding error. The subtraction itself may be correctly rounded while the overall algorithm is inaccurate.
- Why does relative spacing stay roughly constant for normal numbers?** Increasing the exponent scales both the represented number and adjacent spacing by the same power of two, giving approximately constant relative precision.
- Do optimized GPU libraries always produce the most accurate result?** No. They target documented precision and performance modes. A fast Tensor Core path may use reduced-precision inputs or internal arithmetic; choose a stricter mode or higher precision when error requirements demand it.

8. Comparison Tables

Property	Integer	Fixed point	Floating point
Representation	Exact whole values	Integer with implied scale	Significand $\times$ base ^{exponent}
Range	Fixed by bit width	Fixed by width and scale	Very large dynamic range

Property	Integer	Fixed point	Floating point
Precision	Exact within range	Uniform absolute spacing	Roughly uniform relative spacing for normals
Overflow behavior	Language/hardware dependent	Same as integer storage	Infinity or max value depending on operation/mode
Typical GPU use	Indices, counters	DSP and quantized inference	ML, simulation, graphics

Value class	Exponent field	Fraction field	Meaning
Zero	All zeros	All zeros	$+0$ or -0
Subnormal	All zeros	Nonzero	Tiny value without implicit leading 1
Normal	Neither extreme	Any	Ordinary finite value
Infinity	All ones	All zeros	Signed infinity
NaN	All ones	Nonzero	Invalid/undefined result marker

9. Common Mistakes

- Assuming floating point stores decimal digits internally.
- Using exact equality for results of nontrivial computation.
- Choosing one fixed epsilon for both values near zero and values near $1e20$.
- Assuming more threads change only speed, never numerical order.
- Treating every result difference as evidence of a race.
- Ignoring NaN and infinity in validation code.
- Assuming a compiler may freely reassociate arithmetic without a fast-math-style permission.

10. Edge Cases / Special Cases

- $\text{NaN} \neq \text{NaN}$, so NaN-aware checks are necessary.
- Signed zeros compare equal but can produce differently signed infinities.
- Overflow can happen in an intermediate even if the algebraically simplified final result is finite.
- Subnormals have gradually decreasing significant precision.
- Fast math may replace correctly rounded operations with approximations and may flush subnormals.
- Comparing large values requires relative tolerance; comparing near zero requires absolute tolerance.

11. How to Explain in Interview

"Floating point stores a sign, significand, and exponent, so it offers huge dynamic range with finite relative precision. Most values are rounded, arithmetic is not associative, and GPUs may change reduction order or use FMA. I choose the format and library compute mode from the required error bound, then validate with scale-aware tolerances rather than exact equality."

12. Quick Revision Notes

- Normal value: $(-1)^s \times 1.f \times 2^{(e-bias)}$.

- Exponent controls range; significand controls precision.
- Special values: signed zero, subnormal, infinity, NaN.
- Default rounding is usually nearest, ties to even.
- Addition is commutative under ordinary conditions but not associative.
- FMA uses one final rounding.
- Interview trap: machine epsilon is not a universal absolute tolerance.

13. Practice Tasks

1. In Python or C++, print $0.1 + 0.2$, 0.3 , and their difference at high precision.
2. Find the next representable FP32 value after 1 , $1,000$, and $1,000,000$; compare the gaps.
3. Evaluate the same array sum left-to-right, right-to-left, and as a balanced tree.
4. Write a relative-plus-absolute floating-point comparison function with NaN handling.
5. Inspect a float's sign, exponent, and fraction bits using `std::bit_cast` or Python's `struct` module.
6. Compare a dot product with and without an FMA-capable library routine.

14. Final Cheat Sheet

Item	Remember
Core definition	Finite approximation using sign, significand, and exponent
Why it matters	Rounding, special values, and order affect GPU correctness
Most asked	0.1 , non-associativity, NaN, subnormals, FMA
Main comparison	Floating point gives wide range; fixed point gives uniform spacing
One-line answer	Floating point trades exactness for wide dynamic range, so GPU code must control precision, order, and tolerance.

FP16, FP32, and FP64

1. Overview

Definition

FP16, FP32, and FP64 are IEEE-style binary floating-point formats occupying 16, 32, and 64 bits. They differ in range, precision, storage cost, memory traffic, and hardware throughput.

Why it matters

On GPUs, smaller types can move more values per memory transaction and can unlock specialized matrix instructions. Larger types preserve more information and range but cost memory, bandwidth, energy, and often throughput. The fastest type that violates the error budget is not an optimization.

Where they are used

- **FP16:** neural-network training/inference, graphics, bandwidth-limited data.

- **FP32:** default scientific and graphics work, ML accumulators, general GPU kernels.
- **FP64:** scientific simulation, numerical solvers, computational finance, and verification where error bounds demand it.

Why interviewers ask

This tests whether you can connect a data type to precision, range, arithmetic intensity, accelerator hardware, and workload requirements instead of saying “lower precision is faster.”

2. Core Idea

Intuition and analogy

Choosing a format is like choosing a measuring instrument. A 1 mm ruler is compact and adequate for furniture layout; a micrometer is necessary for machining. Carrying a micrometer everywhere is expensive, but using a ruler for a bearing tolerance is wrong.

Small example

Suppose a model stores one billion parameters:

```
FP16: 2 GB
FP32: 4 GB
FP64: 8 GB
```

Ignoring metadata and optimizer state, moving from FP32 to FP16 halves parameter traffic and may allow a larger batch. It also reduces representable range and significant precision, so accumulation often remains FP32.

Step-by-step format choice

1. Establish the acceptable output error and required dynamic range.
2. Identify storage, input, multiplication, accumulation, and output types separately.
3. Check the target GPU's actual throughput for each type.
4. Use an optimized library mode that maps to supported hardware.
5. Compare against a higher-precision reference on representative and adversarial inputs.
6. Profile end-to-end; conversion and memory overhead can erase theoretical gains.

3. Important Subtopics

3.1 Binary16 (FP16)

- **Meaning:** 1 sign, 5 exponent, and 10 stored fraction bits; 11 effective significand bits for normals.
- **Why it matters:** Roughly three to four decimal digits of precision and a maximum finite magnitude around 6.55×10^4 make scaling important.
- **Example:** Gradients smaller than the representable range may become zero without loss scaling.
- **Interview angle:** FP16 multiplication with FP32 accumulation is common; “FP16 operation” does not imply every stage is FP16.

3.2 Binary32 (FP32)

- **Meaning:** 1 sign, 8 exponent, 23 fraction bits; 24 effective significand bits.
- **Why it matters:** Around seven decimal digits of precision and a wide range make it the common general-purpose GPU type.
- **Example:** Store activations in FP16 but reduce dot products into FP32.
- **Interview angle:** FP32 is not exact and can still suffer cancellation or overflow.

3.3 Binary64 (FP64)

- **Meaning:** 1 sign, 11 exponent, 52 fraction bits; 53 effective significand bits.
- **Why it matters:** Around 15–16 decimal digits of precision suits sensitive numerical work.
- **Example:** Long-time integration in a physical simulation may need FP64 to control drift.
- **Interview angle:** Consumer and data-center GPUs can have very different FP64 throughput ratios; never assume it is simply half FP32 speed.

3.4 BF16 and TF32

- **Meaning:** BF16 uses FP32-like exponent range with fewer fraction bits. TF32 is a hardware/library compute format commonly used for matrix operations with FP32-range inputs and reduced multiply precision, typically accumulating more accurately.
- **Why it matters:** Range and precision can be traded independently. BF16 often avoids FP16's narrow-range loss-scaling problems.
- **Example:** A neural network may train with BF16 activations and FP32 accumulation.
- **Interview angle:** Storage type, multiply precision, and accumulator precision must be named separately.

3.5 Storage precision versus compute precision

- **Meaning:** Inputs may be stored in one type, multiplied in another effective precision, and accumulated in a wider type.
- **Why it matters:** It often delivers most of the bandwidth and Tensor Core benefit while protecting long sums.
- **Example:** $\text{FP16} \times \text{FP16} \rightarrow \text{FP32 accumulate} \rightarrow \text{FP16 output}$.
- **Interview angle:** Ask what “uses FP16” actually means.

3.6 Hardware and library support

- **Meaning:** Throughput depends on architecture, instruction shape, alignment, dimensions, and library configuration.
- **Why it matters:** A nominally supported type may fall back to slower instructions if shapes or layouts are unsuitable.
- **Example:** A matrix multiply with aligned dimensions may use Tensor Cores, while an awkward tiny matrix is dominated by launch overhead.
- **Interview angle:** Prefer cuBLAS/cuDNN for standard dense operations and inspect profiler/library logs before writing a custom kernel.

4. Real-World Example

During transformer training, weights and activations may be stored in FP16 or BF16, Tensor Cores multiply low-precision tiles, FP32 accumulators build dot products, and optimizer state or master weights remain FP32. An automatic mixed-precision framework selects eligible operations, while loss scaling protects small FP16 gradients. This division works because matrix multiplication tolerates lower input precision better than reductions, normalization statistics, and weight updates.

5. Diagrams / Mental Models

```

Memory/storage      Multiply      Accumulate      Output
FP16 weights -----> low-precision lanes --> FP32 registers ----> FP16/FP32
FP16 activations ----/

Bandwidth saved      Tensor Core speed      error protected

```

Format	Bits (sign/exponent/fraction)	Approx. decimal precision	Approx. max finite	Bytes/value
FP16	1 / 5 / 10	3–4 digits	6.55e4	2
FP32	1 / 8 / 23	6–9 digits	3.40e38	4
FP64	1 / 11 / 52	15–17 digits	1.80e308	8

6. Common Interview Questions

#	Question and clear answer	Key points expected	Common mistake
1	What is the main difference among FP16, FP32, and FP64? Bit allocation changes precision, range, storage, bandwidth, and hardware throughput.	Connect numerical and performance trade-offs.	Saying only “number of decimal places.”
2	Why can FP16 be faster on a GPU? It halves traffic versus FP32 and can use high-throughput matrix hardware when supported.	Bandwidth plus specialized compute.	Promising 2× speed for every kernel.
3	Why accumulate FP16 products in FP32? A sum may contain thousands of terms; a wider accumulator reduces rounding and prevents small terms from disappearing as quickly.	Distinguish product inputs from accumulator.	Assuming FP32 accumulation restores information already lost in FP16 inputs.
4	When is FP64 necessary? When conditioning, dynamic range, long accumulation, conservation laws, or required tolerances exceed FP32 capability.	Accuracy requirement and validation.	Choosing it merely because it is “safer.”
5	Why is FP64 sometimes much slower? Many GPUs dedicate far fewer execution resources to FP64, and double values double FP32 traffic.	Architecture-specific throughput and bandwidth.	Assuming all GPUs have the same ratios.

#	Question and clear answer	Key points expected	Common mistake
6	What advantage does BF16 have over FP16? It has an FP32-like exponent range, reducing overflow/underflow risk, but fewer significand bits.	Range versus precision.	Calling BF16 more precise than FP16 in every sense.
7	What is TF32? A matrix-compute mode that keeps FP32-like range but uses reduced multiply precision with wider accumulation on supported hardware.	It is not simply an ordinary 19-bit storage type.	Equating it with full IEEE FP32 arithmetic.
8	Does converting FP32 data to FP16 always improve performance? No; conversion overhead, unsupported operations, small workloads, and non-bandwidth bottlenecks can remove the gain.	Profile end-to-end.	Reasoning from byte width alone.
9	How do you choose precision? Define error requirements, test representative worst cases against a high-precision reference, then benchmark supported library modes.	Correctness before performance.	Selecting from intuition only.
10	What should remain high precision in ML training? Common candidates are reductions, normalization statistics, loss-sensitive operations, gradient accumulation, and optimizer/master weights.	Operation-specific policy.	Forcing the entire model into one type.

7. Deep-Dive Questions

- Why can BF16 train better than FP16 despite fewer fraction bits?** Training often needs dynamic range more urgently than fine mantissa precision. BF16 represents tiny and large magnitudes similarly to FP32, while stochastic optimization may tolerate noisier low bits.
- Can FP64 always fix an unstable algorithm?** No. It delays error but does not change poor conditioning or catastrophic formulations. Reformulating the algorithm is often more effective.
- How can low precision increase effective GPU occupancy?** Smaller values reduce register and shared-memory footprint in some kernels, potentially allowing more resident warps, though allocation granularity and compiler choices decide the actual result.
- Why might a library choose different kernels for the same matrix multiply?** Dimensions, strides, alignment, workspace, determinism requirements, input type, compute mode, and hardware all affect the best tiling and instruction path.
- How do you verify Tensor Core use?** Check library configuration and profiler instruction/throughput metrics; do not infer it only from using FP16 inputs.

8. Comparison Tables

Criterion	FP16	FP32	FP64
Memory use	Lowest	Medium	Highest

Criterion	FP16	FP32	FP64
Range	Narrow	Wide	Very wide
Precision	Low	Medium	High
Typical GPU throughput	Often highest	High	Hardware-dependent, often lower
Common role	Inputs/storage	General compute/accumulation	Sensitive scientific compute
Main risk	Overflow, underflow, rounding	Accumulation/cancellation	Cost and false confidence in unstable algorithms
Format	Strong point	Weak point	Common use
FP16	More fraction precision than BF16	Narrow exponent range	Inference and scaled training
BF16	FP32-like exponent range	Coarse significand	Training with fewer range issues
TF32 compute	Fast FP32-range matrix path	Reduced multiply precision	Accelerated FP32 deep-learning GEMM

9. Common Mistakes

- Treating storage, multiply, accumulator, and output precision as one choice.
- Assuming FP16 means exactly twice the speed of FP32.
- Ignoring the target GPU's very different FP64 capabilities.
- Using FP64 to hide an unstable algorithm.
- Comparing only average model accuracy and missing rare overflow or outlier cases.
- Assuming a low-precision input guarantees specialized hardware use.
- Forgetting conversion, packing, alignment, and launch overhead.

10. Edge Cases / Special Cases

- FP16's largest finite value is small enough that ordinary ML intermediate values can overflow.
- BF16 has wide range but coarse spacing, so small updates to large values may vanish.
- Some operations are internally promoted; others remain in the input type.
- Library defaults and precision policies may change, so explicitly record required behavior and benchmark the deployed configuration.
- Tiny or irregular problems may be faster in FP32 because optimized low-precision paths have setup or shape requirements.

11. How to Explain in Interview

"FP16, FP32, and FP64 trade memory and throughput for range and significand precision. On GPUs I choose storage, multiply, and accumulator types separately—often low-precision inputs with FP32 accumulation—and use optimized libraries to reach hardware matrix units. I validate against a higher-precision reference because the right type is determined by the workload's error budget, not by speed alone."

12. Quick Revision Notes

- FP16: 5 exponent, 10 fraction bits; fast and compact but narrow range.
- FP32: 8 exponent, 23 fraction bits; general GPU default.
- FP64: 11 exponent, 52 fraction bits; accurate but potentially expensive.
- BF16 prioritizes range; FP16 has more fraction precision.
- Low-precision multiply plus FP32 accumulation is a common pattern.
- Interview trap: a type's peak throughput is not application speed.

13. Practice Tasks

1. Convert a range of values to FP16 and report relative error, overflow, and underflow.
2. Benchmark FP16, FP32, and FP64 vector operations on the available GPU; classify compute- versus bandwidth-bound behavior.
3. Multiply matrices with an optimized library using different compute modes and compare with an FP64 reference.
4. Find a sum where FP16 accumulation fails but FP32 accumulation succeeds.
5. Measure whether odd matrix dimensions prevent or reduce the expected low-precision speedup.
6. Create an error-versus-runtime chart for three precisions on one realistic workload.

14. Final Cheat Sheet

Item	Remember
Core definition	16-, 32-, and 64-bit floating formats with different range and precision
Why it matters	Type affects traffic, accelerator use, throughput, and error
Most asked	Why FP16 is fast, why accumulate in FP32, when FP64 is needed
Main comparison	FP16: compact; FP32: balanced; FP64: precise and costly
One-line answer	Use the lowest precision that meets a measured error bound, with wider accumulation and optimized libraries where appropriate.

Numerical Stability

1. Overview

Definition

Numerical stability describes how an algorithm controls errors introduced by finite-precision inputs and arithmetic. A stable algorithm produces the exact answer to a nearby problem, or at least keeps rounding error proportional to the unavoidable sensitivity of the problem.

Why it matters

Two formulas can be mathematically equivalent in real arithmetic but behave very differently on a GPU. Parallelism, fast-math approximations, reduced precision, and enormous problem sizes amplify this difference.

Where it is used

Stable algorithms are essential in linear solvers, machine-learning losses, softmax, normalization, probability calculations, signal processing, geometry, simulations, and any long iterative computation.

Why interviewers ask

Interviewers want candidates to distinguish **problem conditioning** from **algorithm stability**, recognize cancellation, and propose a reformulation instead of blindly switching to FP64.

2. Core Idea

Intuition and analogy

Imagine measuring the height of a small box by subtracting two large altitude readings. Even if both readings are individually accurate to a centimeter, their difference may be only a few centimeters and dominated by measurement error. The input method—not the subtraction instruction alone—is fragile.

Small example: softmax

The direct formula is:

$$\text{softmax}(x_i) = \exp(x_i) / \sum \exp(x_j)$$

If $x = [1000, 1001]$, direct exponentiation overflows. Subtracting the maximum gives:

$$m = 1001 \\ \text{softmax}(x_i) = \exp(x_i - m) / \sum \exp(x_j - m)$$

Now the exponent arguments are $[-1, 0]$. The exact mathematical ratio is unchanged, but the machine computation is safe.

Step-by-step stability analysis

1. Identify the expected input range and exceptional values.
2. Determine whether the mathematical problem is sensitive to small input changes.
3. Trace large, tiny, nearly equal, or alternating intermediates.
4. Locate overflow, underflow, cancellation, and long reductions.
5. Reformulate, rescale, widen selected operations, or use a stable library routine.
6. Validate relative, absolute, and application-level error on adversarial inputs.

3. Important Subtopics

3.1 Conditioning versus stability

- **Meaning:** Conditioning belongs to the problem; stability belongs to the algorithm.
- **Why it matters:** No algorithm can recover highly sensitive information absent from the inputs, but an unstable algorithm adds avoidable error.
- **Example:** Solving a nearly singular linear system is ill-conditioned even with a stable solver.
- **Interview angle:** Higher precision can help but does not change the condition number.

3.2 Catastrophic cancellation

- **Meaning:** Subtracting nearly equal approximations cancels leading digits and exposes low-order error.
- **Why it matters:** Relative error in the small result may be enormous.
- **Example:** Use `log1p(x)` instead of `log(1+x)` for tiny `x`, because `1+x` may round to `1`.
- **Interview angle:** The subtraction can be correctly rounded while the formula is still unstable.

3.3 Scaling and normalization

- **Meaning:** Rescale values so intermediates stay in a safe numerical range.
- **Example:** Subtract the maximum before softmax; scale a vector before computing its norm.
- **Why it matters:** Prevents overflow/underflow without changing the intended result.
- **Interview angle:** Explain why the transformation preserves the mathematics.

3.4 Stable special functions

- **Meaning:** Functions such as `log1p`, `expm1`, and `hypot` are designed for difficult regions.
- **Example:** `expm1(x)` accurately computes `exp(x)-1` for small `x`.
- **Why it matters:** Standard or vendor math libraries often solve the edge case better than an open-coded formula.
- **Interview angle:** Use established primitives rather than inventing approximations.

3.5 Stable factorizations and solvers

- **Meaning:** Algorithm choice affects error growth in linear algebra.
- **Example:** QR is usually more stable than solving least squares through normal equations, which squares the condition number.
- **Why it matters:** cuSOLVER or another tuned solver exposes algorithms with different accuracy/performance properties.
- **Interview angle:** "Use a library" is incomplete; choose the right factorization and inspect status/conditioning.

3.6 Fast math versus accuracy

- **Meaning:** GPU fast-math modes can enable approximate reciprocal, square root, transcendental operations, reassociation, and altered subnormal handling.
- **Why it matters:** They may be safe for graphics or tolerant ML kernels and unsafe for sensitive scientific calculations.
- **Example:** An approximate exponential may be acceptable inside inference if end-to-end error is validated.
- **Interview angle:** Fast math is an explicit contract trade-off, not a universal optimization flag.

4. Real-World Example

A GPU backend implements cross-entropy loss. A naive version computes softmax probabilities and then takes `log`, risking overflow in `exp` and underflow to zero before `log`. A stable fused library kernel uses `log-sum-exp`:

$$\log \sum \exp(x_i) = m + \log \sum \exp(x_i - m), \text{ where } m = \max(x)$$

$$\text{loss} = \log \sum \exp(x_i) - x_{\text{target}}$$

The fused implementation also avoids storing the entire probability array, reducing global-memory traffic. Here the optimized library improves both numerical behavior and performance.

5. Diagrams / Mental Models

```

Input error -----> [problem conditioning] -----> unavoidable output sensitivity
                    +
Rounding error --> [algorithm stability] -----> avoidable error growth
  
```

```

Naive softmax:  x -> exp(x) -> sum -> divide
                  ^
                  can overflow

Stable softmax: x -> max -> subtract -> exp -> sum -> divide
                  values <= 0
  
```

6. Common Interview Questions

#	Question and clear answer	Key points expected	Common mistake
1	What is numerical stability? An algorithm is stable when rounding and input perturbations do not grow much beyond the problem's inherent sensitivity.	Error behavior of an algorithm.	Defining it only as "does not crash."
2	What is the difference between conditioning and stability? Conditioning describes sensitivity of the mathematical problem; stability describes error introduced by the method.	Problem versus algorithm.	Saying FP64 fixes ill-conditioning.
3	What is catastrophic cancellation? Nearly equal subtraction removes leading significant digits and magnifies prior relative error.	Give a concrete example.	Saying subtraction is inherently inaccurate.
4	How do you compute softmax safely? Subtract the maximum before exponentiation, then normalize.	Shift invariance and overflow avoidance.	Clipping arbitrary values without explaining changed semantics.

#	Question and clear answer	Key points expected	Common mistake
5	Why use $\log_{1p}(x)$? It preserves small x when evaluating $\log(1+x)$, where $1+x$ may round to 1.	Stable specialized primitive.	Assuming the compiler always transforms it.
6	Does FP64 guarantee a correct answer? No; it reduces rounding but cannot repair wrong formulas, severe conditioning, races, or invalid inputs.	Precision is one tool, not proof.	Treating more bits as correctness.
7	Why can parallelization change stability? It changes reduction order, grouping, use of FMA, and sometimes the selected algorithm.	Non-associativity and tree depth.	Claiming parallel algorithms are always less accurate.
8	How would you test numerical stability? Compare with a trusted higher-precision reference across normal and adversarial cases using application-relevant error metrics.	Worst cases, invariants, tolerances.	Testing one random input.
9	When is fast math acceptable? When the documented semantic changes satisfy the workload's error and special-value requirements and profiling shows a benefit.	Explicit validation and measurement.	Enabling it globally because it is faster.
10	How can a library improve stability? It can use stable factorizations, scaling, compensated/wider reductions, and carefully designed special functions.	Libraries embody algorithmic as well as hardware optimization.	Assuming every library mode prioritizes maximum accuracy.

7. Deep-Dive Questions

- What is backward stability?** A backward-stable algorithm returns an answer equal to the exact solution for a slightly perturbed input. It is powerful because output error can then be related to the problem's condition number.
- Why do normal equations worsen least-squares conditioning?** Forming $A^T A$ roughly squares the condition number, amplifying sensitivity and rounding error. QR or SVD avoids that specific damage at additional cost.
- How can you compute a vector norm without intermediate overflow?** Scale by the largest absolute component or use a stable *hypot*-style iterative scheme, then rescale the result.
- Why is log-sum-exp stable?** Subtracting the maximum makes all exponent arguments non-positive, so the largest exponential is 1; the common factor is restored analytically by adding the maximum in log space.
- Can deterministic computation still be numerically unstable?** Yes. Determinism repeats the same result; it says nothing about closeness to the correct result.

8. Comparison Tables

Concept	Conditioning	Numerical stability
Property of	Mathematical problem	Chosen algorithm/implementation
Main question	Do small input changes alter the true answer greatly?	Does computation amplify avoidable error?
Improved by more precision?	Sometimes mitigated, not fundamentally changed	Often, but reformulation may help more
Example	Nearly singular system	Normal equations for least squares
Naive form	Stable alternative	Failure avoided
$\log(1+x)$	$\log1p(x)$	Loss of tiny x
$\exp(x) - 1$	$\expm1(x)$	Cancellation near zero
Direct softmax	Subtract-max softmax	Exponential overflow
$\sqrt{x^2 + y^2}$	$\text{hypot}(x, y)$ or scaled norm	Square overflow/underflow
Normal equations	QR/SVD	Squared conditioning

9. Common Mistakes

- Confusing a stable algorithm with an accurate answer for an ill-conditioned problem.
- Increasing precision before reformulating a fragile expression.
- Computing an unstable intermediate and trying to repair it afterward.
- Using only average relative error, which behaves poorly around zero.
- Comparing against an FP32 implementation as if it were exact ground truth.
- Enabling fast math without checking NaN, infinity, subnormal, and rounding behavior.
- Reimplementing stable primitives already supplied by math or GPU libraries.

10. Edge Cases / Special Cases

- Relative error is undefined or misleading when the true result is zero; use absolute error or a combined metric.
- A stable method can still return a large forward error for an ill-conditioned input.
- Clamping prevents overflow but changes the mathematical function and gradients.
- Fusing operations may improve rounding and traffic but change intermediate observability and exact bits.
- Stable algorithms sometimes cost more work; the correct choice follows the application's error contract.

11. How to Explain in Interview

"Numerical stability is about preventing the algorithm from amplifying finite-precision error beyond the problem's inherent conditioning. I look for cancellation, extreme intermediates, and long reductions, then use stable transformations such as subtract-max softmax, $\log1p$, scaling, or a better factorization. On GPUs I prefer tuned library kernels and validate their precision mode against a higher-precision reference."

12. Quick Revision Notes

- Conditioning: sensitivity of the problem.
- Stability: error growth caused by the algorithm.
- Cancellation exposes earlier error.
- Reformulation usually beats blindly adding precision.
- Key stable tools: scaling, log space, `log1p`, `expm1`, QR/SVD, wider accumulation.
- Interview trap: deterministic does not mean accurate.

13. Practice Tasks

1. Implement naive and subtract-max softmax; test inputs near the format's overflow limit.
2. Compare `log(1+x)` with `log1p(x)` for decreasing values of `x`.
3. Solve a least-squares problem using normal equations and QR; vary the condition number.
4. Implement naive and scaled Euclidean norms using very large and very small components.
5. Enable and disable a GPU fast-math option; measure error and speed for a transcendental-heavy kernel.
6. Build adversarial tests containing equal values, extreme magnitudes, cancellation, NaN, and infinity.

14. Final Cheat Sheet

Item	Remember
Core definition	Control avoidable error growth in finite-precision computation
Why it matters	Equivalent formulas can have radically different GPU behavior
Most asked	Conditioning vs stability, cancellation, stable softmax, fast math
Main comparison	Better formulation often helps more than more precision
One-line answer	A stable GPU algorithm keeps rounding error controlled through reformulation, scaling, and appropriate library precision.

Accumulation Error

1. Overview

Definition

Accumulation error is the rounding error that builds while many values are added, multiplied and added, or otherwise combined into a running result. It appears in sums, dot products, matrix multiplication, reductions, histograms, means, variances, and gradient aggregation.

Why it matters

GPUs perform enormous reductions in parallel. Floating-point addition is non-associative, so the reduction tree, thread scheduling, atomics, and accumulator type all influence the result. A one-ULP error per step can become important across millions of terms, especially when magnitudes differ or signs cancel.

Where it is used

- Dot products and matrix multiplication
- Neural-network gradients and optimizer statistics
- Parallel sums, means, norms, and variances
- Monte Carlo simulation
- Image and signal aggregation
- Distributed all-reduce operations

Why interviewers ask

This topic connects floating-point theory with actual GPU reduction design. Interviewers expect trade-offs among naive summation, pairwise reduction, wider accumulation, compensation, determinism, atomics, and optimized library routines.

2. Core Idea

Intuition and analogy

Imagine a cash register that records only whole rupees. If its balance is ₹1,000,000, repeatedly adding fractions of a rupee changes nothing because every addition rounds away. Adding the small amounts together first may create several whole rupees that survive when added to the large balance.

Small example

In limited precision:

```
large = 10,000,000
small = 0.25

(((large + small) + small) + small) + small  -> may remain 10,000,000
large + (small + small + small + small)      -> 10,000,001
```

The real sums are identical. The grouping determines when rounding occurs.

Step-by-step GPU reduction

```
Input:      a0  a1  a2  a3  a4  a5  a6  a7
Level 1:    s0      s1      s2      s3
Level 2:      t0          t1
Level 3:          result
```

1. Threads load separate elements.
2. Nearby elements are combined into partial sums.
3. The number of active partials halves at each tree level.
4. Each level rounds its results.
5. A final block or kernel combines block partials.

6. A different tree may produce slightly different low bits, but a balanced tree usually has better error growth than a long serial chain.

3. Important Subtopics

3.1 Sequential summation

- **Meaning:** Add each value to one running accumulator.
- **Why it matters:** Simple, but error can grow roughly with the number of terms in a worst-case analysis.
- **Example:** One thread loops over an entire array.
- **Interview angle:** It is also too serial for a large GPU reduction.

3.2 Pairwise or tree summation

- **Meaning:** Recursively add pairs of similarly sized partial results.
- **Why it matters:** Tree depth is logarithmic and often reduces error while exposing parallelism.
- **Example:** Warp, block, and grid reductions.
- **Interview angle:** Explain that speed and accuracy can improve together, though exact results differ from a CPU loop.

3.3 Wider accumulation

- **Meaning:** Convert inputs to a wider type for the sum.
- **Why it matters:** It reduces rounding and increases dynamic range at a cost determined by hardware.
- **Example:** FP16 or BF16 products accumulated in FP32.
- **Interview angle:** It cannot restore precision already lost when inputs were rounded.

3.4 Compensated summation

- **Meaning:** Track low-order information lost during addition. Kahan and Neumaier summation are common forms.
- **Example:** Kahan keeps a compensation term representing bits that fell off the running sum.
- **Why it matters:** It can substantially improve long or cancellation-heavy sums.
- **Interview angle:** More instructions and dependencies may reduce GPU throughput; use when the error requirement justifies it.

3.5 Atomic accumulation and nondeterminism

- **Meaning:** Many threads update one result with atomics, but their arrival order is generally unspecified.
- **Why it matters:** The operation can be race-free yet produce different last bits across runs.
- **Example:** `atomicAdd` into a shared gradient bin.
- **Interview angle:** Atomic correctness does not imply floating-point reproducibility.

3.6 Dot products and FMA

- **Meaning:** A dot product combines multiplication error and addition error. FMA computes each multiply-add with one rounding.
- **Why it matters:** GEMM spends most of its work in such accumulations.
- **Example:** cuBLAS may use specialized matrix hardware with a documented accumulator type.
- **Interview angle:** Ask about input, product, and accumulator precision separately.

3.7 Variance algorithms

- **Meaning:** The formula $E[x^2] - E[x]^2$ subtracts nearby large values when variance is small.
- **Why it matters:** It can produce large relative error or even a small negative result.
- **Example:** Welford's online algorithm maintains mean and squared deviations more stably and has a parallel merge formula.
- **Interview angle:** A better accumulation algorithm matters more than merely changing reduction syntax.

4. Real-World Example

A distributed training job computes gradients on several GPUs. Each GPU performs local reductions, and NCCL combines buffers across devices. Local kernels, chunking, collective topology, and device count can all change the addition order. Training usually needs statistical convergence rather than bitwise-identical gradients, so high-throughput FP32 accumulation may be appropriate. A scientific validation run may instead require a fixed reduction tree or reproducible summation mode.

5. Diagrams / Mental Models

```
Serial chain: ((((((a+b)+c)+d)+e)+f)+g)+h
Depth:      O(n)

Balanced tree: ((a+b)+(c+d)) + ((e+f)+(g+h))
Depth:      O(log n)
```

Accuracy/cost ladder for a sum

```
low cost -----> higher cost
native accumulator -> wider accumulator -> pairwise + wide -> compensated/exact
```

6. Common Interview Questions

#	Question and clear answer	Key points expected	Common mistake
1	What is accumulation error? Rounding error introduced and propagated while combining many terms.	It depends on order, magnitudes, signs, and format.	Describing only overflow.
2	Why does summation order matter? Floating-point addition rounds each intermediate and is not associative.	Large-plus-small and cancellation examples.	Saying addition is nondeterministic by definition.
3	Why is pairwise summation useful on GPUs? It exposes parallelism and reduces reduction depth, often improving the error bound.	Balanced tree and $O(\log n)$ depth.	Claiming it returns the exact real sum.

#	Question and clear answer	Key points expected	Common mistake
4	What is Kahan summation? A compensated algorithm that carries an estimate of low-order error lost at each step.	Better accuracy, extra operations/dependency.	Calling the compensation an exact remainder in all cases.
5	Why use FP32 accumulation for FP16 inputs? The wider sum retains small contributions longer and has greater range.	Mixed input/accumulator types.	Assuming input quantization error disappears.
6	Can atomics be correct but nondeterministic? Yes. Atomicity prevents lost updates, but varying floating addition order changes rounded bits.	Race freedom versus reproducibility.	Calling the variation a data race.
7	How does FMA help a dot product? Each product-plus-sum has one rounding instead of separate multiply and add roundings.	Better local accuracy and throughput.	Saying one FMA makes the whole dot product exact.
8	How would you compute variance stably? Use Welford's algorithm or a stable two-pass method instead of $E[x^2] - E[x]^2$.	Cancellation and parallel merge.	Clamping a negative variance as the only fix.
9	How do you make a GPU reduction reproducible? Fix the partition and tree, avoid unordered atomics, control algorithms and precision, and accept the performance cost.	Deterministic order.	Assuming a synchronization barrier orders independent blocks.
10	When should you use a library reduction or GEMM? For standard operations unless unsupported semantics or proven fusion benefits justify custom code.	Tuned tiling, hardware instructions, documented compute types.	Rewriting GEMM for a routine workload.

7. Deep-Dive Questions

- What error growth is expected for serial versus pairwise summation?** Under standard assumptions, naive serial summation has a worst-case bound proportional to $n \cdot u$, while balanced pairwise summation is closer to $\log_2(n) \cdot u$, where u is unit roundoff. Actual error also depends on data.
- Why can sorting by magnitude improve a sum?** Adding values of similar magnitude first prevents tiny values from being immediately swallowed by a large accumulator. Sorting costs $O(n \log n)$ and changes memory behavior, so pairwise or binned methods are often more practical.
- Can a reduction be both fast and bitwise reproducible?** Yes for a fixed architecture and launch scheme using a fixed tree, but strict portability across device counts and architectures may require more

expensive reproducible accumulators or specified arithmetic.

4. **How are Welford states merged in parallel?** Combine each partition's count, mean, and sum of squared deviations using the mean difference and counts. This permits a tree reduction without reverting to the unstable one-pass formula.
5. **What is exact or binned accumulation?** Terms are accumulated into exponent-indexed bins or a wide fixed-point superaccumulator so order does not change the rounded result. It provides strong reproducibility at considerable storage and instruction cost.

8. Comparison Tables

Method	Accuracy	Parallelism	Extra cost	Good use
Serial naive	Lowest for difficult long sums	Poor	Minimal	Tiny, benign sums
Pairwise/tree	Usually good	Excellent	Reduction coordination	Default GPU reduction
Wider accumulator	Better	Excellent	Type-dependent	Low-precision inputs
Kahan/Neumaier	High for many workloads	More difficult	Extra arithmetic/state	Strict error budget
Exact/binned	Strongest/reproducible	Possible but costly	High	Auditing or reproducibility-critical work
Property	Atomic reduction	Fixed tree reduction		
Race-free	Yes with proper atomics	Yes with correct synchronization		
Order	Usually unspecified	Designed and repeatable		
Reproducible bits	Usually no	Often on a fixed configuration		
Contention	Can be high	Hierarchical aggregation reduces it		

9. Common Mistakes

- Assuming race-free atomics imply deterministic floating results.
- Accumulating low-precision data in the same low-precision type by default.
- Using one thread for accuracy and losing all GPU parallelism.
- Believing pairwise summation eliminates all error.
- Comparing a GPU tree with a serial CPU loop using exact equality.
- Computing variance as $E[x^2] - E[x]^2$ for nearly constant data.
- Adding compensation without measuring its instruction and dependency cost.

10. Edge Cases / Special Cases

- Alternating signs can make relative error huge when the true sum is near zero.
- All-positive sums avoid cancellation but can still lose small terms.

- Overflow can occur in partial sums even when later negative terms make the final result finite.
- Reordering may change NaN propagation and the sign of zero.
- A fixed kernel can change results if block count, input length, GPU count, or library algorithm changes.
- Error metrics for a near-zero true sum should include absolute error.

11. How to Explain in Interview

“Accumulation error comes from rounding every partial sum. On a GPU, the reduction tree and atomic arrival order affect the answer because addition is not associative. My default is a balanced library reduction with a wider accumulator—such as FP16 inputs into FP32—and I use a fixed tree or compensation only when reproducibility or the measured error requires it.”

12. Quick Revision Notes

- Long sums lose low-order information.
- Balanced trees are parallel and usually more accurate than serial chains.
- Wider accumulation helps; it cannot repair rounded inputs.
- FMA rounds once per multiply-add.
- Atomics prevent lost updates, not order-dependent rounding.
- Use Welford or two-pass variance.
- Interview trap: different low bits do not prove a race.

13. Practice Tasks

1. Sum the same adversarial array serially, pairwise, in random orders, and with Kahan summation.
2. Implement a CUDA block reduction with FP32 and FP64 accumulators; compare error and runtime.
3. Run an `atomicAdd` reduction repeatedly and inspect bitwise result variation.
4. Implement parallel Welford state merging and compare it with $E[x^2] - E[x]^2$.
5. Compare a custom dot product against cuBLAS and a high-precision CPU reference.
6. Vary block size and grid size; explain why output bits may change.

14. Final Cheat Sheet

Item	Remember
Core definition	Rounding error built while combining many values
Why it matters	GPU reductions reorder non-associative operations
Most asked	Pairwise sum, Kahan, wider accumulators, atomic nondeterminism
Main comparison	Pairwise is the usual speed/accuracy default; compensation costs more
One-line answer	Control accumulation error with a balanced reduction, suitable accumulator width, and a defined reproducibility policy.

Mixed Precision

1. Overview

Definition

Mixed precision uses multiple numerical formats in one computation, assigning lower precision to tolerant, high-throughput work and higher precision to sensitive storage, reductions, updates, or corrections.

Why it matters

It can reduce memory usage and bandwidth, increase Tensor Core throughput, and fit larger problems on a GPU while retaining accuracy close to a higher-precision baseline. It is an operation-level design, not simply “cast the whole program to FP16.”

Where it is used

- Neural-network training and inference
- Dense and sparse linear algebra
- Iterative refinement for linear systems
- Scientific simulation with selective FP64
- Graphics and image pipelines
- Communication compression in distributed training

Why interviewers ask

Interviewers test whether you understand loss scaling, master weights, autocasting, accumulator precision, error validation, and why some operations must remain high precision.

2. Core Idea

Intuition and analogy

An architect may use a quick sketch to explore a building but uses precise measurements for the foundation. Mixed precision spends numerical accuracy where errors have the greatest downstream effect.

Small example: matrix multiply

```
A stored as FP16
B stored as FP16
A × B performed on Tensor Cores
partial sums accumulated as FP32
C stored as FP16 or FP32, depending on its next use
```

The inputs provide bandwidth and multiply throughput benefits. The FP32 accumulator protects the long dot products.

Step-by-step ML training flow

1. Keep a model's sensitive state, commonly master weights and optimizer state, in FP32.
2. Cast eligible forward and backward operations to FP16 or BF16.

3. Accumulate matrix products and reductions in FP32 where supported.
4. For FP16, multiply the loss by a scale so small gradients remain representable.
5. Backpropagate, then unscale gradients.
6. Detect NaN or infinity; skip the update and reduce the scale if overflow occurred.
7. Apply the optimizer update to FP32 state and repeat.

3. Important Subtopics

3.1 Automatic mixed precision (AMP/autocast)

- **Meaning:** A framework chooses lower precision for approved operations and retains higher precision for sensitive ones.
- **Why it matters:** Operator policies encode known behavior and reduce unsafe manual casting.
- **Example:** GEMM and convolution may run low precision; some reductions or exponentials remain FP32.
- **Interview angle:** Autocast chooses operation precision; it does not replace validation.

3.2 Loss scaling

- **Meaning:** Multiply the loss and therefore gradients by a scale before FP16 backpropagation, then divide gradients by the scale before the optimizer step.
- **Why it matters:** It shifts tiny gradients into FP16's representable range without changing the mathematical update after unscaling.
- **Example:** Dynamic scaling grows the scale after successful steps and shrinks it after overflow.
- **Interview angle:** Loss scaling addresses underflow, not rounding already present in forward activations.

3.3 Master weights and optimizer state

- **Meaning:** Maintain FP32 weights for updates while low-precision copies feed forward/backward computation.
- **Why it matters:** A small update may round to zero if directly added to an FP16 weight.
- **Example:** Adam's moment estimates and parameter update remain FP32.
- **Interview angle:** Extra FP32 state reduces some memory savings.

3.4 Operation sensitivity

- **Meaning:** Different operations tolerate precision differently.
- **Why it matters:** Large matrix multiplications often tolerate low-precision inputs; reductions, normalization, exponentials, small differences, and solver residuals may not.
- **Example:** Compute layer-normalization statistics in FP32.
- **Interview angle:** Give a per-operation policy, not a global type rule.

3.5 Iterative refinement

- **Meaning:** Solve approximately in low precision, compute residuals in high precision, and solve correction equations until convergence.
- **Why it matters:** It can approach high-precision accuracy while most expensive factorization/solve work uses faster low precision.
- **Example:** $\mathbf{r} = \mathbf{b} - \mathbf{A}\mathbf{x}$ in FP64, correction solve in FP32, update $\mathbf{x}$ in FP64.
- **Interview angle:** Success depends on conditioning and the low-precision solver's quality.

3.6 Tensor Cores and optimized libraries

- **Meaning:** Specialized units perform matrix multiply-accumulate at high throughput for supported types and shapes.
- **Why it matters:** cuBLAS, cuDNN, and framework libraries handle tiling, layouts, epilogues, and instruction selection.
- **Example:** Use library GEMM with an explicit compute type instead of writing a general matrix multiply.
- **Interview angle:** Confirm actual instruction use and end-to-end speed with profiling.

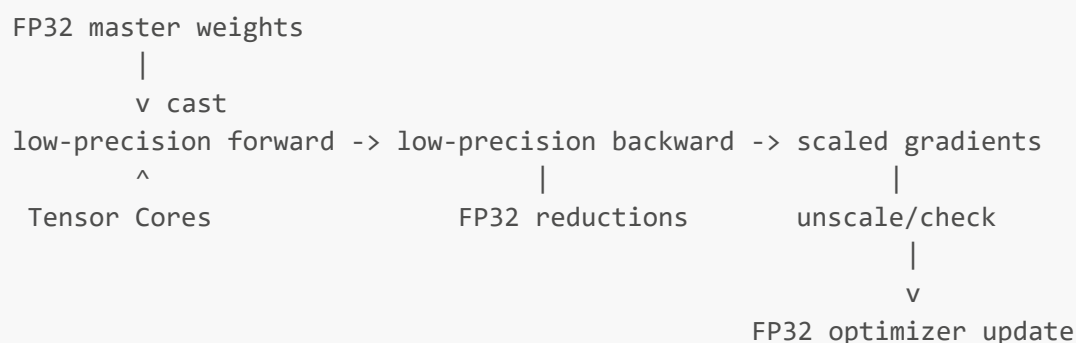
3.7 Error monitoring and fallback

- **Meaning:** Compare against a baseline, check finite values, monitor convergence, and selectively promote unstable operations.
- **Why it matters:** Mixed-precision failures may be rare and data-dependent.
- **Example:** Promote a problematic normalization or final reduction to FP32 without abandoning low precision everywhere.
- **Interview angle:** Recovery should be targeted and evidence-based.

4. Real-World Example

A vision model trains on a GPU using BF16 convolution and matrix multiplication with FP32 accumulation. Batch-normalization statistics and optimizer state stay FP32. BF16's exponent range usually removes the need for FP16-style loss scaling, while its compact storage and Tensor Core path improve throughput. The team validates final accuracy, checks for non-finite gradients, and profiles data loading so the GPU speedup is not hidden by an input bottleneck.

5. Diagrams / Mental Models



Use low precision where work is tolerant and dominant.
Use high precision at numerical choke points:

```

inputs -> [large GEMM] -> [normalization] -> [large GEMM] -> [loss/reduction]
         FP16/BF16      FP32 stats          FP16/BF16      FP32
  
```

6. Common Interview Questions

#	Question and clear answer	Key points expected	Common mistake
1	What is mixed precision? Using different formats for different data and operations to improve performance while meeting accuracy requirements.	Storage, multiply, accumulation, update can differ.	Saying it is simply FP16 training.
2	Why does mixed precision speed up GPUs? Smaller data reduces memory traffic and supported matrix operations use higher-throughput specialized units.	Bandwidth and compute.	Guaranteeing a gain for every workload.
3	What is loss scaling? Scale the loss before backpropagation, unscale gradients before updating, and adjust the scale when overflow is detected.	Protect FP16 gradients from underflow.	Applying scaled gradients directly to weights.
4	Why keep FP32 master weights? Tiny updates may disappear when applied to coarse FP16 weights; FP32 preserves them.	Update precision.	Assuming the low-precision model copy alone is enough for all training.
5	Which operations should remain high precision? Sensitive reductions, normalization statistics, loss calculations, residuals, and optimizer updates are common candidates.	Workload-specific validation.	Memorizing an immutable list.
6	Why might BF16 not require loss scaling? Its exponent range is similar to FP32, so gradient underflow is less severe, though precision is still lower.	Range versus significand precision.	Saying BF16 cannot underflow.
7	What is automatic mixed precision? Framework logic that casts operations according to tested policies and manages scaling utilities.	It is a policy mechanism.	Treating AMP as a correctness guarantee.
8	How do you validate mixed precision? Compare convergence and outputs to a high-precision baseline, check non-finite values, use worst-case inputs, and profile.	Numerical plus performance criteria.	Checking only one training step.
9	What is iterative refinement? Use a low-precision solve for speed and high-precision residual/correction steps to recover accuracy.	Multiple precisions serve different roles.	Claiming it converges for every ill-conditioned system.
10	When should you avoid mixed precision? When required error, conditioning, unsupported operations, conversion overhead, or lack of accelerator benefit makes it unsafe or slower.	Evidence-driven decision.	Saying only scientific workloads cannot use it.

7. Deep-Dive Questions

1. **Why can dynamic loss scaling skip an optimizer step?** A non-finite scaled gradient means the unscaled gradient cannot be trusted. Skipping preserves model state; lowering the scale makes the next attempt less likely to overflow.
2. **What condition allows iterative refinement to work?** Roughly, the product of the system's condition number and low-precision unit roundoff must be sufficiently below one, with details depending on the algorithm and precision combination.
3. **Can mixed precision change convergence even without NaNs?** Yes. Quantized activations, gradients, and reductions add noise or bias; final task metrics and multiple seeds may be needed, not only finite-value checks.
4. **Why fuse mixed-precision operations?** Fusion can avoid intermediate writes and conversions and can keep accumulators in registers, but the fused kernel must preserve the intended precision at sensitive stages.
5. **What limits the theoretical Tensor Core speedup?** Memory bandwidth, small or incompatible shapes, conversion, non-matrix operations, launch overhead, data input, and synchronization can dominate end-to-end runtime.

8. Comparison Tables

Approach	Compute/storage policy	Speed potential	Numerical risk	Typical use
Full FP32	Mostly FP32	Baseline	Moderate	General GPU compute
Full FP16	Mostly FP16	High	High for sums/range	Selected inference
Mixed FP16/FP32	FP16 work, FP32 sensitive stages	High	Managed with scaling/policy	Training/inference
Mixed BF16/FP32	BF16 work, FP32 sensitive stages	High	Coarse precision, good range	Training
FP32/FP64 refinement	Low-precision solve, high-precision residual	Workload-dependent	Conditioning-limited	Linear solvers
Static loss scale		Dynamic loss scale		
Fixed by developer		Adjusted from overflow feedback		
Simple and predictable		Adapts during training		
Can waste range or overflow		Adds checks and occasional skipped steps		
Suitable when gradient range is known		Safer general FP16-training default		

9. Common Mistakes

- Casting every tensor to FP16 and calling it mixed precision.
- Forgetting to unscale gradients before clipping or updating.

- Keeping normalization statistics or long reductions in too little precision.
- Assuming BF16 is always more accurate than FP16 because its range is wider.
- Reporting kernel peak speed instead of end-to-end application speed.
- Ignoring extra memory from master weights and optimizer state.
- Falling back to full FP32 when promoting one unstable operation would suffice.

10. Edge Cases / Special Cases

- Gradient clipping must operate on unscaled gradients or use scale-aware thresholds.
- Overflow detection must cover all relevant gradient partitions in distributed training.
- Small models may be launch-bound and gain little from lower precision.
- Irregular dimensions may reduce specialized matrix-unit efficiency.
- A stable forward pass can still produce underflowing backward gradients.
- Checkpointing must preserve the intended master and optimizer types for a faithful restart.

11. How to Explain in Interview

"Mixed precision assigns low precision to high-volume tolerant work and higher precision to sensitive stages. A common GPU pattern is FP16 or BF16 matrix inputs, Tensor Core multiplication, FP32 accumulation, and FP32 optimizer state. For FP16 training I use dynamic loss scaling, then validate convergence and profile end-to-end rather than assuming low precision is automatically safe or faster."

12. Quick Revision Notes

- Mixed precision is an operation-level policy.
- Separate storage, multiply, accumulator, and update types.
- FP16 training often needs loss scaling; BF16 usually needs it less.
- Keep sensitive reductions and persistent updates wider.
- Tensor Core use depends on hardware, shapes, layouts, and library mode.
- Interview trap: AMP automates casting, not correctness proof.

13. Practice Tasks

1. Train a small model in FP32 and AMP; compare time, memory, convergence, and non-finite counts.
2. Implement static then dynamic loss scaling on a toy FP16 gradient.
3. Profile a library GEMM to confirm specialized matrix instructions are used.
4. Promote one normalization/reduction operation at a time and observe error changes.
5. Implement iterative refinement for a small linear system using low-precision solves and high-precision residuals.
6. Create a precision map listing every tensor's storage, compute, accumulation, and output type.

14. Final Cheat Sheet

Item	Remember
Core definition	Multiple precisions assigned according to operation sensitivity
Why it matters	Gains throughput and memory efficiency while preserving accuracy

Item	Remember
Most asked	Loss scaling, master weights, AMP, FP32 accumulation, Tensor Cores
Main comparison	FP16 needs range management; BF16 trades fraction precision for range
One-line answer	Put low precision on dominant tolerant work and high precision at reductions, state updates, and error checks.

Quantization

1. Overview

Definition

Quantization maps values from a large or continuous numerical set, commonly FP32, into a smaller discrete set such as INT8 or INT4. A scale and sometimes a zero point connect real values to integer codes.

For affine quantization:

```
q = clamp(round(x / scale) + zero_point, q_min, q_max)
x_approx = scale * (q - zero_point)
```

Why it matters

Quantization reduces model size and memory traffic and can use high-throughput integer or low-bit matrix hardware. Its cost is information loss from rounding and clipping, plus metadata and conversion overhead.

Where it is used

- Neural-network inference on GPUs, CPUs, phones, and edge devices
- Large-language-model weight compression
- Communication of gradients or activations
- Compressed embeddings and vector search
- Signal and image processing
- Integer matrix multiplication in optimized inference libraries

Why interviewers ask

Interviewers expect you to derive scale/zero-point behavior, explain calibration, compare PTQ and QAT, choose quantization granularity, and connect accuracy to actual kernel speed.

2. Core Idea

Intuition and analogy

Quantization is like replacing an analog ruler with one that has only 256 marks. A wider measuring range prevents clipping but places marks farther apart. A narrower range gives finer resolution but cannot represent outliers. Calibration chooses the useful compromise.

Small example

Map real values from $[-1, 1]$ to signed INT8 values $[-127, 127]$ using symmetric quantization:

```
scale = 1 / 127
zero_point = 0

x = 0.50
q = round(0.50 / scale) = round(63.5) = 64
x_approx = 64 × scale ≈ 0.50394
```

The reconstructed value is close but not identical. A value 1.5 is clipped to 127 , reconstructing as 1.0 .

Step-by-step quantized matrix multiplication

1. Calibrate or learn scales for input A and weights B .
2. Quantize them to integer values qA and qB .
3. Multiply integer values and accumulate into a wider integer, commonly INT32.
4. Convert using the combined scale $scale_A \times scale_B$.
5. Add bias in a compatible scale or higher precision.
6. Apply activation and requantize for the next quantized layer, preferably in a fused library epilogue.

3. Important Subtopics

3.1 Symmetric and asymmetric quantization

- **Meaning:** Symmetric quantization centers the range around zero, often with $zero_point = 0$. Asymmetric quantization uses a nonzero zero point to fit an offset range.
- **Why it matters:** Symmetric arithmetic is simpler; asymmetric mapping uses codes better for skewed/nonnegative data.
- **Example:** ReLU activations in $[0, 6]$ benefit from an unsigned or asymmetric range.
- **Interview angle:** Zero must be represented exactly for padding and sparse behavior.

3.2 Scale and zero point

- **Meaning:** Scale sets real spacing between codes; zero point identifies the integer code for real zero.
- **Why it matters:** Incorrect metadata makes arithmetic numerically meaningless.
- **Example:** For range $[x_{min}, x_{max}]$, an affine scale is roughly $(x_{max} - x_{min}) / (q_{max} - q_{min})$, followed by a rounded and clamped zero point.
- **Interview angle:** Scale must be positive, and endpoints/rounding require careful handling.

3.3 Quantization granularity

- **Meaning:** One scale may cover a tensor, channel, row, group, or block.

- **Why it matters:** Finer granularity adapts to different ranges and improves accuracy but increases metadata and kernel complexity.
- **Example:** Per-output-channel weight scales handle filters with different magnitudes.
- **Interview angle:** Per-channel weights are common; activation granularity depends on runtime cost and hardware support.

3.4 Post-training quantization (PTQ)

- **Meaning:** Quantize a trained model without full retraining, using calibration data to choose ranges.
- **Why it matters:** Fast deployment path with low engineering cost.
- **Example:** Collect activation histograms, choose clipping thresholds, then export INT8 inference.
- **Interview angle:** Calibration data must represent deployment data.

3.5 Quantization-aware training (QAT)

- **Meaning:** Simulate rounding and clipping during training using fake-quantization nodes, allowing weights to adapt.
- **Why it matters:** Often recovers accuracy when PTQ fails, especially at low bit widths.
- **Example:** Forward pass uses simulated INT8 values while trainable parameters and updates remain floating point.
- **Interview angle:** The straight-through estimator commonly approximates gradients through the nondifferentiable round operation.

3.6 Static and dynamic activation quantization

- **Meaning:** Static quantization fixes activation ranges from calibration; dynamic quantization computes ranges at runtime.
- **Why it matters:** Dynamic scaling adapts to each input but adds reduction, synchronization, and conversion overhead.
- **Example:** Weight-only LLM quantization avoids activation quantization overhead while reducing weight bandwidth.
- **Interview angle:** Dynamic quantization is useful only if accuracy gains exceed its runtime cost.

3.7 Outliers and clipping

- **Meaning:** Rare large values expand the scale, making most values use only a small portion of available codes.
- **Why it matters:** Clipping outliers improves bulk resolution but introduces saturation error.
- **Example:** Keep outlier channels in FP16 while quantizing the rest to INT8/INT4.
- **Interview angle:** Min/max calibration is simple but often overly sensitive to outliers.

3.8 Integer accumulation and requantization

- **Meaning:** Low-bit products accumulate into a wider type, then are rescaled for output.
- **Why it matters:** The accumulator must not overflow, and rounding/saturation in requantization affects accuracy.
- **Example:** `INT8 × INT8 -> INT32 accumulate -> INT8 or FP16 output.`
- **Interview angle:** Derive the output multiplier from input, weight, and output scales.

3.9 Optimized quantized libraries

- **Meaning:** Inference engines and GEMM libraries pack weights, select supported low-bit instructions, fuse dequantization/bias/activation, and tune layouts.
- **Why it matters:** A smaller model is not faster if every layer dequantizes to FP32 and writes large intermediates.
- **Example:** Use a supported cuBLASLt, cuDNN, or inference-engine path for standard INT8 matrix/convolution operations.
- **Interview angle:** Quantize only when a real end-to-end kernel path exists on the deployment GPU.

4. Real-World Example

An LLM is memory-bandwidth-bound during single-user decoding. Weight-only INT4 quantization reduces bytes fetched per token. A fused GPU kernel loads packed INT4 weights, dequantizes groups into registers, multiplies them by FP16 activations, and accumulates in FP32. Groupwise scales improve accuracy, while fusion prevents a full dequantized weight matrix from being written to memory. The result can be faster because it attacks the true bottleneck: weight bandwidth.

5. Diagrams / Mental Models

```
Real line:      -1.0  ... -0.2 ... 0 ... 0.7 ... 1.0
                | map using scale + zero point
Integer codes: -127 ... -25 ... 0 ... 89 ... 127
                | approximate inverse mapping
Reconstructed: discrete values, with rounding and clipping error
```

Quantization error = rounding error + clipping error

wider range -> less clipping, coarser steps
narrower range -> finer steps, more clipping risk

6. Common Interview Questions

#	Question and clear answer	Key points expected	Common mistake
1	What is quantization? Mapping values to fewer discrete codes using scaling, rounding, and clipping.	Compression and approximate arithmetic.	Calling it lossless data compression.
2	What are scale and zero point? Scale is the real-value step per code; zero point is the code representing real zero.	Give quantize/dequantize equations.	Treating zero point as an arbitrary bias added after inference.
3	Symmetric versus asymmetric quantization? Symmetric centers at zero and simplifies	Simplicity versus range utilization.	Saying asymmetric is always more

#	Question and clear answer	Key points expected	Common mistake
	arithmetic; asymmetric shifts the code range to fit skewed values better.		accurate and free.
4	PTQ versus QAT? PTQ calibrates a finished model; QAT exposes training to simulated quantization so parameters adapt.	Cost/accuracy trade-off.	Saying QAT performs all training in integer arithmetic.
5	Per-tensor versus per-channel quantization? Per-tensor uses one scale; per-channel uses multiple scales for better local fit at metadata/kernel cost.	Granularity trade-off.	Ignoring the channel axis and hardware support.
6	Why does quantization improve GPU performance? It reduces memory traffic and can use higher-throughput low-bit instructions when kernels are supported and fused.	Performance is conditional on the execution path.	Assuming smaller files guarantee faster inference.
7	Why accumulate INT8 products in INT32? Products and long sums exceed INT8 range; a wider accumulator preserves them.	Accumulator overflow analysis.	Requantizing after every multiply.
8	What is calibration? Running representative data to estimate useful activation ranges or distributions for scales and clipping thresholds.	Representative dataset and objective.	Calibrating with arbitrary random data.
9	How do outliers hurt quantization? They enlarge the scale, leaving coarse resolution for most values, or they get clipped if the range is tightened.	Rounding-versus-clipping trade-off.	Always using raw min/max.
10	When can quantization be slower? Unsupported shapes, packing/dequantization overhead, dynamic range computation, small batches, or non-quantized bottlenecks can dominate.	Benchmark deployed end-to-end system.	Comparing theoretical operation counts only.

7. Deep-Dive Questions

- How is a quantized dot product corrected for nonzero zero points?** Expand $\sum (q_A - z_A)(q_B - z_B)$ into $\sum q_A q_B - z_B \sum q_A - z_A \sum q_B + n \cdot z_A \cdot z_B$. Libraries may precompute weight sums or use instructions/layouts that reduce this overhead.
- How do you choose a clipping threshold from a histogram?** Minimize a calibration objective—such as reconstruction error, divergence, or task loss—over candidate thresholds instead of automatically preserving extreme min/max values.
- Why is per-group INT4 common for large models?** INT4 needs little weight bandwidth, while a separate scale for each small group limits the damage from heterogeneous weight ranges. The metadata and dequantization cost are amortized across matrix work.
- What is the accumulator overflow bound for signed INT8 dot products?** A worst-case term is roughly $127 \times 127 = 16129$; multiply by dot-product length and compare with INT32 limits. Zero-

point corrections and actual ranges also matter.

5. **Why can QAT use gradients despite rounding?** Training commonly uses fake quantization in the forward pass and a straight-through estimator that substitutes a useful approximate derivative through rounding, usually with clipping-aware behavior.

8. Comparison Tables

Criterion	PTQ		QAT	
Requires retraining	No or minimal tuning		Yes	
Deployment effort	Lower		Higher	
Accuracy at very low bits	Often lower		Usually better	
Data need	Representative calibration set		Training/fine-tuning data	
Best use	Fast INT8 deployment		Sensitive models or INT4/low-bit targets	
Granularity	Accuracy	Metadata	Kernel simplicity	
Per-tensor	Lowest	Lowest	Highest	
Per-channel	Better	Moderate	Hardware-dependent	
Per-group/block	Often strong	Higher	More dequantization work	
Representation	Dynamic range		Precision pattern	Typical GPU use
INT8 affine	Fixed calibrated range		Uniform absolute steps	Inference GEMM/convolution
INT4 groupwise	Very compact, group-scaled		Coarse within each group	Weight-only LLM inference
FP8	Floating exponent and fraction		Rough relative precision	Supported training/inference paths

9. Common Mistakes

- Quantizing without a kernel that can consume the quantized layout directly.
- Using unrepresentative calibration data.
- Preserving extreme min/max values and sacrificing resolution for typical values.
- Applying one scale to channels with very different ranges.
- Forgetting bias and accumulator scale relationships.
- Assuming INT8 multiplication means INT8 accumulation.
- Measuring model-file size but not latency, throughput, memory, and task accuracy.

10. Edge Cases / Special Cases

- A constant tensor makes $x_{\max} - x_{\min} = 0$; scale selection must handle it explicitly.
- Real zero should map exactly when padding, sparsity, or zero-preserving operations depend on it.
- Rounding tie behavior can affect reproducibility across tooling.
- INT32 accumulators can overflow for sufficiently long or poorly bounded dot products.

- Dynamic shapes or batches can invalidate static calibration assumptions.
- NaN and infinity require a policy before quantization because ordinary integer codes cannot preserve their semantics.

11. How to Explain in Interview

“Quantization maps real values to low-bit codes using a scale and often a zero point. It saves bandwidth and can unlock integer matrix units, but adds rounding and clipping error. I choose symmetric or asymmetric mapping and tensor/channel/group granularity from the data, calibrate on representative inputs or use QAT, accumulate products widely, and verify that the deployment library has a fused low-bit path.”

12. Quick Revision Notes

- $q = \text{clamp}(\text{round}(x/s) + z); x \approx s(q - z)$.
- Symmetric: simpler, often $z=0$; asymmetric: better fit for shifted ranges.
- PTQ is cheap; QAT usually preserves more accuracy.
- Finer granularity improves fit but costs metadata and kernel work.
- Low-bit product needs a wider accumulator.
- Performance requires supported packed and fused kernels.
- Interview trap: quantized storage alone does not guarantee quantized execution.

13. Practice Tasks

1. Implement symmetric and asymmetric INT8 quantize/dequantize functions and plot reconstruction error.
2. Compare min/max calibration with percentile clipping on outlier-heavy data.
3. Quantize a small matrix per-tensor and per-channel; compare output error.
4. Derive and test the zero-point correction terms for an integer dot product.
5. Calculate the maximum safe INT8 dot-product length for an INT32 accumulator under worst-case assumptions.
6. Profile a fused quantized library layer versus quantize–dequantize around an FP32 layer.

14. Final Cheat Sheet

Item	Remember
Core definition	Map real values to low-bit discrete codes using scale and zero point
Why it matters	Reduces bandwidth/storage and can accelerate supported matrix kernels
Most asked	Symmetric vs asymmetric, PTQ vs QAT, calibration, per-channel scales
Main comparison	Finer granularity improves accuracy but adds metadata and kernel cost
One-line answer	Quantization is useful when calibrated low-bit error is acceptable and the deployed library executes it without costly materialization.

Overflow and Underflow

1. Overview

Definition

Overflow occurs when a result's magnitude exceeds the largest representable finite value. **Underflow** occurs when a nonzero result is smaller in magnitude than the normal range; it may become subnormal or round to zero.

Why it matters

Overflow can produce infinity, NaN in later operations, or saturation/wraparound for integer arithmetic. Underflow can erase gradients, probabilities, or physical quantities. GPU fast modes and low-precision formats make both problems especially relevant.

Where it is used

- Exponentials, softmax, and probability products
- FP16/BF16 training and inference
- Norms, distances, and variance
- Long products and reductions
- Quantized integer accumulators
- Physical simulation and iterative solvers

Why interviewers ask

This tests whether you understand range, subnormals, infinity/NaN propagation, stable scaling, loss scaling, log-domain computation, and the difference between floating and integer overflow.

2. Core Idea

Intuition and analogy

A calculator with a fixed display may show “too large” when digits exceed the screen and 0 when a result is too tiny to display. Floating point adds an exponent, but its exponent field is still finite. Values outside that exponent range cannot remain ordinary finite numbers.

Small example

In FP16, the largest finite value is about 65504:

```
400 × 400 = 160000 -> overflow in FP16
```

For a norm, the final answer $\sqrt{400^2 + 300^2} = 500$ is representable, but directly squaring the inputs in FP16 overflows. A scaled or *hypot*-style calculation avoids the bad intermediate.

Step-by-step safe range reasoning

1. Identify the format's normal, subnormal, and maximum finite ranges.
2. Bound intermediate values, not only final outputs.

3. Find exponentials, products, squares, reciprocals, and long accumulators.
4. Reformulate using scaling, normalization, log space, or wider intermediates.
5. Decide how NaN, infinity, and zero should be handled.
6. Validate boundary values and inspect non-finite counts during execution.

3. Important Subtopics

3.1 Floating-point overflow

- **Meaning:** A rounded result is beyond the finite range and normally becomes signed infinity under round-to-nearest IEEE behavior.
- **Why it matters:** Later operations such as $\infty - \infty$ or $0 \times \infty$ produce NaN.
- **Example:** Direct `exp(1000)` in FP32 overflows.
- **Interview angle:** Prevent the large intermediate rather than merely checking the final output.

3.2 Gradual underflow and subnormals

- **Meaning:** Results below the smallest normal value can be represented with reduced precision as subnormals before reaching zero.
- **Why it matters:** It avoids an abrupt gap but relative accuracy degrades.
- **Example:** A decaying simulation state passes through subnormal values.
- **Interview angle:** Distinguish smallest normal from smallest positive subnormal.

3.3 Flush-to-zero (FTZ) and denormals-are-zero (DAZ)

- **Meaning:** FTZ converts tiny results to zero; DAZ treats tiny inputs as zero. Exact behavior depends on architecture, operation, compiler, and mode.
- **Why it matters:** These modes can improve throughput or simplify hardware but change tiny-value semantics.
- **Example:** A gradient that would be subnormal becomes exactly zero.
- **Interview angle:** Do not make blanket claims; document and test the actual compute mode.

3.4 Scaling and normalization

- **Meaning:** Multiply or divide by a common factor to keep intermediates representable, then restore the scale analytically.
- **Example:** Subtract the maximum in softmax or divide vector components by the maximum before squaring.
- **Why it matters:** It prevents both overflow and premature underflow.
- **Interview angle:** Demonstrate mathematical equivalence.

3.5 Log-domain computation

- **Meaning:** Replace products with sums of logarithms and use stable log-sum-exp operations.
- **Why it matters:** Products of many probabilities underflow even when their log-likelihood remains useful.
- **Example:** $\log \prod p_i = \sum \log p_i$.
- **Interview angle:** Handle zeros and negative inputs according to domain semantics.

3.6 Loss scaling and gradient range

- **Meaning:** Scale the loss before FP16 backpropagation so gradients do not underflow, then unscale before the update.
- **Why it matters:** FP16's exponent range is narrow.
- **Example:** Dynamic scaling lowers the scale after detecting infinity in gradients.
- **Interview angle:** Too large a scale causes overflow; too small fails to prevent underflow.

3.7 Integer overflow in quantized computation

- **Meaning:** Integer accumulators have fixed bounds and may wrap or saturate depending on instruction/type semantics.
- **Why it matters:** INT8 dot products can overflow INT32 for sufficiently long worst-case vectors.
- **Example:** Bound $n \times \max(|a_i|, |b_i|)$ before choosing the accumulator.
- **Interview angle:** Floating overflow to infinity and integer overflow are different behaviors.

3.8 Detecting and localizing range failures

- **Meaning:** Check finite values, ranges, and counters at meaningful pipeline boundaries.
- **Why it matters:** NaN may appear many operations after the first infinity or zero.
- **Example:** Record maximum absolute activation and first non-finite layer during training.
- **Interview angle:** `isfinite` detects symptoms; range analysis and anomaly localization find the cause.

4. Real-World Example

A GPU computes log-likelihoods for long token sequences. Multiplying token probabilities directly quickly underflows to zero, even in FP64. The implementation instead sums log probabilities. When probabilities originate from logits, a fused log-softmax library kernel subtracts the maximum and computes log-sum-exp without materializing tiny probabilities. This is both more stable and more bandwidth-efficient than separate softmax and log kernels.

5. Diagrams / Mental Models

```
-infinity ... -max finite | normal | subnormal | -0 +0 | subnormal | normal | max
finite ... +infinity
                                gradual underflow zone
```

```
Too large:  finite -> overflow -> infinity -> invalid combination -> NaN
Too small:  normal -> subnormal -> zero      -> information lost
```

```
Unsafe norm:  sqrt(x2 + y2)
Safe idea:    m = max(|x|, |y|)
               m × sqrt((x/m)2 + (y/m)2)
```

6. Common Interview Questions

#	Question and clear answer	Key points expected	Common mistake
1	What is floating-point overflow? A result exceeds the finite range and commonly rounds to signed infinity.	Consequent NaN propagation is possible.	Saying it wraps like an unsigned integer.
2	What is underflow? A nonzero exact result lies below the normal range and becomes subnormal or zero after rounding/mode handling.	Gradual underflow.	Defining every subnormal result as zero.
3	What are subnormal numbers? Tiny values represented without the normal implicit leading 1, giving reduced precision near zero.	Smooth transition toward zero.	Claiming constant relative precision.
4	How do you prevent softmax overflow? Subtract the maximum logit before exponentiation.	Shift invariance.	Clamping exponentials after overflow.
5	How do you prevent probability-product underflow? Work in the log domain and sum log probabilities.	Domain constraints and log-sum-exp.	Adding an arbitrary epsilon without analyzing bias.
6	What is flush-to-zero? A mode that replaces subnormal results with zero, trading tiny-value semantics for implementation/performance advantages.	Mode-dependent GPU behavior.	Assuming all GPU operations always flush.
7	Why can a representable final answer still fail? Intermediate squares, products, exponentials, or sums may overflow/underflow before cancellation or rescaling.	Analyze intermediates.	Checking only output range.
8	How does loss scaling work? Scale loss/gradients up for FP16 representation, detect overflow, then unscale before the optimizer step.	Underflow protection and dynamic adjustment.	Forgetting to unscale.
9	How is integer overflow different? Integer arithmetic may wrap, saturate, or be language-undefined; it does not normally produce IEEE infinity/NaN.	Know operation-specific semantics.	Treating signed integer overflow as portable wraparound in C++.
10	How do you debug NaNs in a GPU pipeline? Check inputs and first non-finite boundary, capture ranges, disable risky fast modes, use anomaly tools, and trace back from NaN to an earlier overflow/invalid operation.	Localize the first failure.	Replacing every NaN with zero and continuing.

7. Deep-Dive Questions

1. **Why can underflow be harmless in some softmax terms?** After subtracting the maximum, extremely negative logits may exponentiate to zero, but their true contribution is negligible relative to the dominant term. It is acceptable only if the resulting error meets the requirement.
2. **How can you compute a product without overflow or underflow?** Accumulate sign and logarithms, or repeatedly rescale mantissas and track exponents separately. The best choice depends on whether an actual product or its logarithm is needed.
3. **Can overflow be avoided by switching only the output type?** No. The arithmetic compute type and intermediate representation must be widened or reformulated before the overflow occurs.
4. **Why do subnormals lose relative precision?** Their exponent is fixed at the minimum scale while the fraction decreases toward zero, so constant absolute spacing represents an increasing fraction of the value.
5. **How should a library API expose range behavior?** Through explicit input, compute, and accumulator types; math/determinism modes; documented special-value behavior; status reporting; and tests at format boundaries. Callers should not infer these from output storage alone.

8. Comparison Tables

Property	Overflow	Underflow
Trigger	Magnitude too large	Nonzero magnitude too small
Floating result	Usually infinity	Subnormal or zero
Main danger	NaN later, unstable control flow	Silent loss of small contributions
Typical fix	Scale down, log space, wider range	Scale up, log space, wider range
ML example	FP16 gradient becomes infinity	FP16 gradient becomes zero

Arithmetic	Common boundary behavior	Important caution
IEEE floating point	Infinity/NaN and subnormals/zero	Compiler/hardware modes can alter details
Saturating integer	Clamps to min/max	Loses excess magnitude but does not wrap
Wrapping integer	Modular wrap	Can reverse sign or become a small value
C++ signed integer	Overflow is undefined behavior	Never rely on wraparound

Technique	Protects against	Example
Scaling	Large and tiny intermediates	Stable norm
Subtract maximum	Exponential overflow	Softmax/log-sum-exp
Log domain	Product underflow/overflow	Sequence likelihood
Wider accumulator	Reduction range failure	FP16 products into FP32
Dynamic loss scaling	Small FP16 gradients	Mixed-precision training

9. Common Mistakes

- Checking only whether the final mathematical result fits.
- Treating infinity as the original bug rather than tracing the operation that produced it.

- Replacing NaN or infinity with zero and hiding corrupted computation.
- Assuming every GPU and math mode preserves subnormals identically.
- Confusing underflow with rounding error at ordinary magnitudes.
- Using loss scaling without overflow detection and unscaling.
- Forgetting integer accumulator bounds in quantized kernels.

10. Edge Cases / Special Cases

- $0 \times \infty$, $\infty - \infty$, and $0/0$ produce NaN under IEEE arithmetic.
- A negative finite value passed to real `sqrt` produces NaN, not overflow.
- Signed overflow behavior differs between floating point and integer languages/instructions.
- Underflow flags, subnormal production, and a result rounded to zero are related but not identical concepts.
- Rescaling by a maximum needs explicit handling when every input is zero or when NaN is present.
- Saturating output conversion can hide an overflowing accumulator; validate before narrowing.

11. How to Explain in Interview

"Overflow means an intermediate exceeds the format's finite range; underflow means a nonzero result enters the subnormal region or becomes zero. On GPUs I bound intermediates, use scaling or log-domain identities, keep sensitive accumulations wider, and account for fast-mode subnormal behavior. I localize the first non-finite value rather than masking NaNs at the output."

12. Quick Revision Notes

- Overflow: too large, commonly infinity in floating point.
- Underflow: below normal range, then subnormal or zero.
- Final range is insufficient; inspect intermediates.
- Subtract-max softmax and log-domain products are must-know transformations.
- FTZ/DAZ behavior is mode and hardware dependent.
- FP16 loss scaling balances underflow against overflow.
- Interview trap: integer overflow does not follow IEEE floating behavior.

13. Practice Tasks

1. Find the largest finite and smallest normal/subnormal values for FP16, FP32, and FP64.
2. Demonstrate a representable vector norm whose naive intermediate squares overflow.
3. Compare direct probability products with summed log probabilities for increasing sequence length.
4. Implement subtract-max log-sum-exp and test large positive and negative inputs.
5. Create a dynamic loss-scale simulation that halves the scale on overflow and grows it after successful steps.
6. Bound an INT8 dot product and select an accumulator type; verify with adversarial maximum inputs.

14. Final Cheat Sheet

Item	Remember
Core definition	Overflow exceeds maximum range; underflow falls below normal range

Item	Remember
Why it matters	Produces infinity, NaN, subnormals, or silent zeros
Most asked	Stable softmax, log domain, subnormals/FTZ, loss scaling
Main comparison	Scale down against overflow; scale up against underflow
One-line answer	Prevent range failures before they occur by bounding intermediates, scaling, using log space, or widening selected operations.

Choosing Optimized Numerical Libraries

The seven topics above determine how to configure and validate optimized libraries. For standard GPU operations, use the established library first:

Workload	Typical optimized library category	Numerical choices to inspect
Dense matrix/vector operations	BLAS / cuBLAS / cuBLASLt	Input type, compute type, accumulator, Tensor Core mode, determinism
Convolution and neural-network primitives	cuDNN or framework backend	Math mode, fused operation, layout, precision policy
Linear systems and factorizations	cuSOLVER	Factorization choice, pivoting, conditioning, refinement
FFTs	cuFFT	Transform precision, scaling convention, accumulated roundoff
Multi-GPU reductions	NCCL or framework collectives	Reduction type, order/reproducibility, accumulator range
Quantized inference	Supported inference engine / low-bit GEMM	Packing, scales, zero points, accumulator width, fused requantization

Use an optimized library when the operation matches its abstraction, because it already handles architecture-specific tiling, vectorized/coalesced access, specialized matrix instructions, and many edge cases. Consider a custom kernel when profiling shows that unsupported fusion, unusual layouts, or domain-specific arithmetic removes substantial data movement. In either case, correctness requires a higher-precision reference, adversarial range tests, an application-level error budget, and profiling on the actual deployment GPU.